



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

Closure

São funções anônimas que podem ser salvas em variáveis ou passadas como parâmetro para funções. Ao contrário de funções closures podem capturar o contexto onde foram criados.

```
fn  inc_v1    (x: u32) -> u32 { x + 1 }

let inc_v2 = |x: u32| -> u32 { x + 1 };
let inc_v3 = |x|           { x + 1 };
let inc_v4 = |x|           x + 1 ;

let a = 10;
let foo = |x| x + a;
```

Ponteiro para Funções

```
fn map (f:fn(&i32) -> T2, v1:&Vec<i32>) -> Vec<i32>{
    let mut v2 = Vec::new();
    for e in v1.iter(){
        v2.push(f(e));
    }
    v2
}
fn main() {
    let v1 = vec![10, 20, 30];
    let v2;

    v2 = map(|x| x + 1, &v1);
    println!("{v2:?}");
}
```

Ponteiro para Funções

```
fn map<T1, T2> (f:fn(&T1) -> T2, v1:&Vec<T1>) -> Vec<T2>{
    let mut v2 = Vec::new();
    for e in v1.iter(){
        v2.push(f(e));
    }
    v2
}

fn main() {
    let v1 = vec![10, 20, 30];
    let v2;
    let v3 = vec!["programacao", "em", "rust"];
    let v4;

    v2 = map(|x| x + 1, &v1);
    println!("{v2:?}");
    v4 = map(|s| s.to_uppercase(), &v3);
    println!("{v4:?}");
}
```

Closure

```
fn twice<F>(f:F, x:i32) -> i32 where
  F: Fn (i32) -> i32 {
    f(f(x))
  }

fn inc (x:i32) -> i32
{
  x + 1
}

fn main() {
  let a = 10;
  println!("{}", twice (inc, 5));
  println!("{}", twice (|x| x + a, 5));
}
```

Closure

```
fn map<F, T1, T2> (f:F, v1:&Vec<T1>) -> Vec<T2> where
  F : Fn(&T1) -> T2 {
    let mut v2 = Vec::new();
    for e in v1.iter(){
      v2.push(f(e));
    }
    v2
  }

fn main() {
  let a = 10;
  let v1 = vec![10, 20, 30];
  let v2:Vec<i32>;

  v2 = map(|x| x + a, &v1);
  println!("{v2:?}");
}
```

Iterator

```
fn main() {  
    let v1 = vec![10, 20, 30];  
    let v:Vec<i32> = v1.iter().map(|x| x + 1).collect();  
    println!("{v:?}");  
}
```